EC200U&EG91xU Series QuecOpen(SDK) Booting&Shutdown Development Guide

LTE Standard Module Series

Version: 1.1

Date: 2023-09-05

Status: Released

At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.
Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China
Tel: +86 21 5108 6236
Email: info@quectel.com

Or our local offices. For more information, please visit:
http://www.quectel.com/support/sales.htm.

For technical support, or to report documentation errors, please visit:
http://www.quectel.com/support/technical.htm.
Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

About the Document

Revision History

Version	Date	Author	Description
-	2022-01-10	JoJo YAN	Creation of the document
1.0	2022-02-25	JoJo YAN	First official release
1.1	2023-09-05	Arvin XU	1. Updated the document name based on the unified QuecOpen naming. 2. Added the applicable modules EG912U-GL and EG915U series.

Contents

Table Index

1 Introduction

Quectel LTE Standard EC200U and EG91xU family (EG912U-GL and EG915U series) modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS system, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see ***document [1]***.

This document is applicable to QuecOpen® solution based on SDK build environment, and provides the booting and shutdown development guide in application layer of EC200U and EG91xU family modules in QuecOpen® solution, including the relevant API and development example.

2 Booting and Shutdown API

2.1. Header File

ql_power.h, the header file of booting and shutdown API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all the header files mentioned in this document are in this directory.

2.2. API Overview

Table 1: API Overview

Function	Description
ql_power_down()	Shuts the module down.
ql_power_reset()	Reboots the module.
ql_get_powerup_reason()	Gets the reason for booting.
ql_get_pwrkey_status()	Gets the PWRKEY status of the module.
ql_pwrkey_callback_register()	Registers the callback function that shuts the module down by pressing PWRKEY.
ql_pwrkey_shutdown_time_set()	Sets the duration of long pressing PWRKEY for shutdown.

2.3. API Description

2.3.1. ql_power_down

This function shuts the module down.

- **Prototype**

```
ql_errcode_power ql_power_down(ql_PowdMode powd_mode)
```

- **Parameter**

powd_mode:
[In] Shutdown modes. See ***Chapter 2.3.1.1*** for details.

- **Return Value**

See ***Chapter 2.3.1.2*** for result codes.

2.3.1.1. ql_PowdMode

The enumeration of shutdown modes:

```
typedef enum
{
    POWD_IMMDLY,
    POWD_NORMAL
}ql_PowdMode;
```

- **Member**

Member	Description
POWD_IMMDLY	Immediate shutdown.
POWD_NORMAL	Normal shutdown.

2.3.1.2. ql_errcode_power

The result codes of booting and shutdown indicate if the function was executed successfully. The cause of error is returned if the execution failed. The enumeration of result codes:

```
typedef enum
{
    QL_POWER_POWD_SUCCESS    = QL_SUCCESS,
    QL_POWER_RESET_SUCCESS = QL_POWER_POWD_SUCCESS,

    QL_POWER_CFW_CTRL_ERR                    =    1|QL_POWER_ERRCODE_BASE,
    QL_POWER_CFW_CTRL_RSP_ERR,
    QL_POWER_CFW_RESET_BUSY,

    QL_POWER_SEMAPHORE_CREATE_ERR        =    5|QL_POWER_ERRCODE_BASE,
    QL_POWER_SEMAPHORE_TIMEOUT_ERR,

    QL_POWER_POWD_EXECUTE_ERR              = 11|QL_POWER_ERRCODE_BASE,
    QL_POWER_POWD_INVALID_PARAM_ERR,

    QL_POWER_RESET_EXECUTE_ERR              = 21|QL_POWER_ERRCODE_BASE,
    QL_POWER_RESET_INVALID_PARAM_ERR,

    QL_POWER_UP_REASON_GET_ERR              = 31|QL_POWER_ERRCODE_BASE,
    QL_POWER_UP_REASON_MEM_NULL_ERR,

    QL_POWER_KEY_CB_NULL_ERR                = 41|QL_POWER_ERRCODE_BASE,
    QL_POWER_KEY_STATUS_GET_ERR,
    QL_POWER_KEY_MEM_NULL_ERR,
    QL_POWER_KEY_SHUTDOWN_TIME_SET_ERR,

    QL_POWER_USB_DETECT_INVALID_PARAM       = 51|QL_POWER_ERRCODE_BASE,
    QL_POWER_USB_DETECT_SAVE_NV_ERR,

}ql_errcode_power;
```

- **Member**

Member	Description
QL_POWER_POWD_SUCCESS	Successful shutdown.
QL_POWER_RESET_SUCCESS	Successful reboot.
QL_POWER_CFW_CTRL_ERR	Failed to execute CFW.
QL_POWER_CFW_CTRL_RSP_ERR	CFW response error.
QL_POWER_CFW_RESET_BUSY	Not currently powered on.
QL_POWER_SEMAPHORE_CREATE_ERR	Failed to create semaphore.

QL_POWER_SEMAPHORE_TIMEOUT_ERR	Waiting for semaphore timeout.
QL_POWER_POWD_EXECUTE_ERR	Failed to shut down.
QL_POWER_POWD_INVALID_PARAM_ERR	Shutdown parameter error.
QL_POWER_RESET_EXECUTE_ERR	Failed to reboot.
QL_POWER_RESET_INVALID_PARAM_ERR	Reboot parameter error.
QL_POWER_UP_REASON_GET_ERR	Failed to get the reason for booting.
QL_POWER_UP_REASON_MEM_NULL_ERR	Booting reason parameter is NULL.
QL_POWER_KEY_CB_NULL_ERR	Callback function of PWRKEY is NULL.
QL_POWER_KEY_STATUS_GET_ERR	Got the incorrect PWRKEY status.
QL_POWER_KEY_MEM_NULL_ERR	PWRKEY status parameter is NULL.
QL_POWER_KEY_SHUTDOWN_TIME_SET_ERR	Set the incorrect duration of long pressing PWRKEY for shutdown.
QL_POWER_USB_DETECT_INVALID_PARAM	Set the incorrect value of USB check parameter.
QL_POWER_USB_DETECT_SAVE_NV_ERR	Failed to save the value of USB check parameter to NVRAM.

2.3.2. ql_power_reset

This function reboots the module.

- **Prototype**

```
ql_errcode_power ql_power_reset(ql_ResetMode reset_mode)
```

- **Parameter**

reset_mode:
[In] Reboot modes. See ***Chapter 2.3.2.1*** for details.

- **Return Value**

See ***Chapter 2.3.1.2*** for result codes.

2.3.2.1. ql_ResetMode

The enumeration of reboot modes:

```
typedef enum
{
    RESET_QUICK,
    RESET_NORMAL
}ql_ResetMode;
```

- **Member**

Member	Description
RESET_QUICK	Immediate reboot.
RESET_NORMAL	Normal reboot.

2.3.3. ql_get_powerup_reason

This function gets the reason for booting.

- **Prototype**

```
ql_errcode_power ql_get_powerup_reason(uint8_t *pwrup_reason)
```

- **Parameter**

**pwrup_reason*:
[Out] Reasons for booting. See ***Chapter 2.3.3.1*** for details.

- **Return Value**

See ***Chapter 2.3.1.2*** for result codes.

2.3.3.1. ql_pwrup_reason

The enumeration of reasons for booting:

```
typedef enum
{
    QL_PWRUP_UNKNOWN,
    QL_PWRUP_PWRKEY,
    QL_PWRUP_PIN_RESET,
    QL_PWRUP_ALARM,
    QL_PWRUP_CHARGE,
    QL_PWRUP_WDG,
    QL_PWRUP_PSM_WAKEUP,
```

```
    QL_PWRUP_PANIC
}ql_pwrup_reason;
```

- **Member**

Member	Description
QL_PWRUP_UNKNOWN	Unknown reason for booting.
QL_PWRUP_PWRKEY	Booting by pressing PWRKEY.
QL_PWRUP_PIN_RESET	Booting by pressing RESET_N.
QL_PWRUP_ALARM	Booting triggered by alarm.
QL_PWRUP_CHARGE	Booting triggered by charging USB.
QL_PWRUP_WDG	Booting triggered by watchdog.
QL_PWRUP_PSM_WAKEUP	Booting woken by PSM.
QL_PWRUP_PANIC	Booting triggered by Dump.

2.3.4. ql_get_pwrkey_status

This function gets the PWRKEY status of the module.

- **Prototype**

```
ql_errcode_power ql_get_pwrkey_status(uint8_t *pwrkey_status)
```

- **Parameter**

**pwrkey_status*:
[Out] PWRKEY status obtained. See ***Chapter 2.3.4.1*** for details.

- **Return Value**

See ***Chapter 2.3.1.2*** for result codes.

2.3.4.1. QL_PWRKEY_STATUS_E

The enumeration of PWRKEY status:

```
typedef enum
{
```

```
    PWRKEY_RELEASE = 0,
    PWRKEY_PRESSED
}QL_PWRKEY_STATUS_E;
```

- **Member**

Member	Description
PWRKEY_RELEASE	Release PWRKEY.
PWRKEY_PRESSED	Press PWRKEY.

2.3.5. ql_pwrkey_callback_register

This function registers the callback function that shuts the module down by pressing PWRKEY. After the module registers the callback function, when you shut it down by pressing PWRKEY, the original shutdown process is terminated and the callback function is executed instead. You need to manually execute *ql_power_down()* to shut the module down after executing your program.

- **Prototype**

```
ql_errcode_power ql_pwrkey_callback_register(ql_pwrkey_callback pwrkey_cb)
```

- **Parameter**

pwrkey_cb:
[In] Callback function that shuts the module down by pressing PWRKEY. See ***Chapter 2.3.5.1*** for details.

- **Return Value**

See ***Chapter 2.3.1.2*** for result codes.

2.3.5.1. ql_pwrkey_callback

This callback function shuts the module down by pressing PWRKEY.

- **Prototype**

```
typedef void (*ql_pwrkey_callback)(void)
```

- **Parameter**

None

- **Return Value**

None

2.3.6. ql_pwrkey_shutdown_time_set

This function sets the duration of long pressing PWRKEY for shutdown.

- **Prototype**

```
ql_errcode_power ql_pwrkey_shutdown_time_set(uint32_t shutdown_time)
```

- **Parameter**

shutdown_time:
[In] Required duration of long pressing PWRKEY for shutdown. Minimum: 420. Unit: ms.

- **Return Value**

See ***Chapter 2.3.1.2*** for result codes.

3 Development Process

This chapter explains how to use the above API for booting and shutdown development and simple debugging in application layer.

3.1. Booting and Shutdown Development Example

3.1.1. Example Description

power_demo.c, the booting and shutdown example file, is provided in *components\ql-application\power* directory of the SDK of EC200U and EG91xU family QuecOpen modules.

Two shutdown and reboot modes are set in *power_demo.c*, which selects different shutdown and reboot modes according to the shutdown and reboot parameters. The entry function is *ql_power_app_init()*, as shown in the following figure:

```
void ql_power_app_init(void)
{
    QlOSStatus err = QL_OSI_SUCCESS;
    ql_task_t power_task = NULL;

    err = ql_rtos_task_create(&power_task, 1024, APP_PRIORITY_NORMAL, "ql_powerdemo", ql_power_demo_thread, NULL, 1);
    if( err != QL_OSI_SUCCESS )
    {
        QL_POWERDEMO_LOG("power demo task created failed[%d]",err);
    }
}
```

power_demo.c contains the example of registering the callback function that shuts the module down by pressing PWRKEY. The entry function is *ql_pwrkey_app_init()*, as shown in the following figure. The example calls the user-registered callback function when you press PWRKEY to shut down the module and sends *QUEC_PWRKEY_SHUTDOWN_START_IND*, the event indicating the start of shutdown, to the task.

```
void ql_pwrkey_app_init(void)
{
    QlOSStatus err = QL_OSI_SUCCESS;

    err = ql_rtos_task_create(&pwrkey_task, 1024, APP_PRIORITY_NORMAL, "ql_pwrkeydemo", ql_pwrkey_demo_thread, NULL, 3);
    require_action(err, return, "pwrkey demo task created failed");

}
```

3.1.2. Example Auto-Start

The above example is not started by default in *ql_init_demo_thread()*, as shown in the following figure:

```
static void ql_init_demo_thread(void *param)
{
    QL_INIT_LOG("init demo thread enter, param 0x%x", param);

/*Caution:If the macro of secure boot and the function are opened, download firmware a
    ..........the secret key cannot be changed forever*/....
#ifdef QL_APP_FEATURE_SECURE_BOOT
    ....//ql_dev_enable_secure_boot();
#endif

#if 0
    ....ql_gpio_app_init();
    ....ql_gpioint_app_init();
#endif

#ifdef QL_APP_FEATURE_LEDCFG
#if !defined(QL_APP_PROJECT_DEF_EC600U) || \
    ...(defined(QL_APP_PROJECT_DEF_EC600U) && !defined(QL_APP_FEATURE_DSSS) && !defined(
    ....ql_ledcfg_app_init();
#endif
#endif

#ifdef QL_APP_FEATURE_LCD
    ....//ql_lcd_app_init();
#endif

    ....//ql_sim_app_init();
    ... ql_power_app_init();
#ifdef QL_APP_FEATURE_PWK
    ... //ql_pwrkey_app_init();
#endif
```

Hibernation/wake-up is enabled when the demo is automatically started, whereas shutdown/reboot is disabled.

3.2. Booting and Shutdown Debugging

3.2.1. Preparation

You can debug the booting and shutdown feature of EC200U and EG91xU family QuecOpen modules on Quectel LTE OPEN EVB.

Download the compiled firmware to the module and connect the USB port of the LTE OPEN EVB to the PC with a USB cable. The USB AP Log Port mainly displays the system debugging information. You can view information of the example through USB AP Log Port after capturing the log with *cooltools*. For details about log capture, see ***document [2]***.

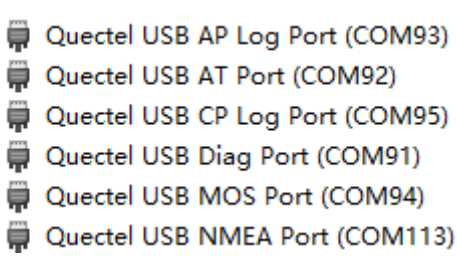

Figure 1: Ports in Device Manager

3.2.2. Debugging

ql_power_app_init() is automatically started after the module is rebooted. In the logs, you can see the differences between execution programs before "immediate shutdown/reboot" and "normal shutdown/reboot". At the same time, you can observe the executed shutdown/reboot with "**Powerdown EXE**" indicating shutdown and "**Reset EXE**" indicating reboot.

Debugging information printed from debug UART port is shown in the figure below:

```
AT get event: 505(EV_CFW_NW_DEREGISTER_RSP)/0x00000000/0x00000000/0x00ff0000
AT: unregistered event
AT get event: 5048(EV_CFW_EXIT_IND)/0x00000001/0x00000000/0x00000000
AT RESPONSE EVENT HANDLED BY ID!
NVM id/4 ops/4 paddr/80002D00 len/2128
ipc: ch4 write len/1 ret/16
IPC ISR status/0x00000008
NVM id/1 ops/4 paddr/80007100 len/2156
ipc: ch4 write len/1 ret/16
RTC unset alarm
RTC write day/7595 hour/8 min/30 sec/59
RTC store nv length/6
[ql_GPIODEMO][ql_gpio_demo_thread, 95] gpio[8] output low-level
[ql_GPIODEMO][ql_gpio_demo_thread, 96] gpio[8] set dir:[1], lvl:[0]
[ql_GPIODEMO][ql_gpio_demo_thread, 102] gpio[8] output low-level
[ql_GPIODEMO][ql_gpio_demo_thread, 103] gpio[8] get dir:[1], lvl:[0]
[ql_GPIODEMO][ql_gpio_demo_thread, 95] gpio[22] output low-level
[ql_GPIODEMO][ql_gpio_demo_thread, 96] gpio[22] set dir:[1], lvl:[0]
[ql_GPIODEMO][ql_gpio_demo_thread, 102] gpio[22] output low-level
[ql_GPIODEMO][ql_gpio_demo_thread, 103] gpio[22] get dir:[1], lvl:[0]
[ql_POWER][ql_power_down, 130] Powerdown EXE
```

```
monitor 0 battery 1/3609
[ql_AUDIO][ql_audio_demo_thread, 39] hello audio demo 4
[osi_event][dog_timer_cb, 758] [kevin]timer run
[ql_AUDIO][ql_audio_demo_thread, 39] hello audio demo 5
[osi_event][dog_timer_cb, 758] [kevin]timer run
tcpip: \n
[ql_AUDIO][ql_audio_demo_thread, 39] hello audio demo 6
[osi_event][dog_timer_cb, 758] [kevin]timer run
[ql_POWER][ql_power_down, 83] Powerdown EXE
```

NOTE

Since running this example causes the module to shut down or reboot, other examples cannot be tested at the same time.

3.2.3. Debugging Through Callback Function

ql_pwrkey_app_init() is automatically started after the module is rebooted. In the logs, you can see that the callback function is triggered by pressing PWRKEY to shut the module down. You need to execute *ql_power_down()* to shut the module down manually after executing your program.

```
switch(event.id)
{
    case QUEC_PWRKEY_SHUTDOWN_START_IND:
        QL_POWERDEMO_LOG("customer process");
        /* do something */
        ql_power_down(POWD_NORMAL);
        break;

    default:
        break;
}
```

4 Appendix References

Table 2: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_CSDK_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 3: Terms and Abbreviations

Abbreviation	Description
AP	Application Processor
API	Application Programming Interface
CFW	Communication Framework
IoT	Internet of Things
NVRAM	Non-Volatile Random Access Memory
PC	Personal Computer
PSM	Power Saving Mode
RTOS	Real-Time Operating System
SDK	Software Development Kit
USB	Universal Serial Bus